

Apuntes

Víctor Manuel Peiró Martínez

FIIS 2A

Bases de Datos

Tabla de contenido

Tema 1 – Introducción A Las Bases de Datos.....	4
1 – Bases de Datos Relacionales.....	4
2 – Sistema Gestor de Bases de Datos	4
3 – Arquitectura de Bases de Datos Relacionales	5
4 – Modelo de BBDD	5
5 – BBDD no Relacionales.....	6
Tema 2 – Modelo Relacional y Algebra Relacional	7
1 – Reglas de Codd	7
2 – Estructura Modelo Relacional.....	8
3 – Esquemas y Atributos del Modelo Relacional	8
4 – Claves: Primarias y Foráneas.....	9
5 – Fases de Diseño BBDD	9
6 – Normalización.....	10
7 – Operaciones de Algebra Relacional	11
Tema 3 – Lenguaje SQL y SQL Avanzado.....	13
1 – Lenguaje SQL: DDL, DML, DCL, TCL	13
2 – Creación y Automatización de Tablas.....	13
3 – Consultas	14
4 – Comando Transaccionales y de Bloqueo.....	15
5 – Vistas. Tipos de Vistas	15
Tema 4 – Acceso a MySQL, Almacenamiento e Indexado	16
1 – Formas de Acceso a BBDD	16
2 – Acceso por Consola.....	16
3 – Interacción Mediante Scripts	16
4 – Interacción a Traves de Lenguajes de Programación	16
5 – Tipos de Almacenamiento	17
6 – Jerarquía	18
7 – Indexado	18
Tema 5 – Administración de Bases de Datos MySQL.....	19
1 – Lenguaje SQL: DCL y DDL.....	19
2 – Usuarios y Privilegios.....	19
3 – Operativa Básica de MySQL	20
4 – Copias de Seguridad.....	20
5 – Restauración de Copias	20
6 – Optimización/Tuning.....	20

Tema 6 – Aspectos Avanzados de Bases de Datos.....	21
1 – Lenguajes Procedurales de la Base de Datos.....	21
2 – Procedimientos Almacenados	21
3 – SQL Injection.....	22
4 – Bases de Datos NoSQL	22
5 – Clasificación de Bases de Datos NoSQL.....	23
6 – MongoDB	23

Tema 1 – Introducción A Las Bases de Datos

1 – Bases de Datos Relacionales

Base de Datos Relacional: Conjunto de elementos de datos con relaciones predefinidas entre ellos organizadas en tablas

Usos:

- Procesamiento de transacciones online
- Data Analíticos

2 – Sistema Gestor de Bases de Datos

Base De Datos: El conjunto de datos

El objetivo primordial de un SGBD es proporcionar eficiencia y seguridad a la hora de extraer o almacenar información en las BBDD.

SGBD: Aplicación que permite definir, crear y mantener la BD y proporcionar un acceso controlado a la misma

Funciones:

1. Creación y Definición de BD
2. Acceso Controlado a los datos
3. Acceso Compartido a la BD
4. Manipulación de los datos
5. Mantener la integridad y consistencia
6. Mecanismos de respaldo y recuperación

Transacción: Una operación única lógica

Toda transacción ha de cumplir una serie de propiedades

Propiedades ACID:

- **Atomicidad:** Toda transacción es o “todo o nada”.
- **Consistencia:** Toda transacción lleva a la base de datos de un estado valido a otro valido
- **Aislamiento (Isolation):** Asegura que la ejecución concurrente de transacciones resulte de manera que parezca que se ejecuten una detrás de otra
- **Durabilidad:** Una vez confirmada la transacción, quedara constante incluso ante caídas del sistema

3 – Arquitectura de Bases de Datos Relacionales

Niveles de Abstracción:

- **Nivel Físico**
 - Nivel más Bajo
 - Describe como se almacenan los datos realmente
- **Nivel Lógico**
 - Describe que datos se almacenan en la base de datos
 - Describe relaciones entre los datos
- **Nivel de Vistas**
 - Nivel más Alto
 - Describe una parte de las bases de datos

Formas de Acceso a BBDD:

- Directamente en el SGBD
- A través de otro programa (Python, Java, PHP, ...)

Las bases de datos pueden estar distribuidas.

Bases De Datos Distribuida: Aquella en que sus datos están distribuidos en varios ordenadores que están interconectados entre si

Sistema Administrador de Bases de Datos Distribuidas: Proporciona a mecanismo de acceso que hace transparente la distribución a los usuarios

4 – Modelo de BBDD

Modelos de BBDD:

- **Modelo Relacional**
 - Utiliza un grupo de tablas para representar los datos y las relaciones entre ellos
 - Cada tabla tiene columnas y cada columna un nombre único
- **Modelo Entidad Relacional**
 - Basado en la percepción del mundo real
 - **Entidad:** Cosa/Objeto de la vida real distinguible entre objetos
 - Las entidades se describen mediante atributos
 - **Relación:** Asociación entre varias entidades

Lenguajes de BBDD:

- **LDD/DDDL – Lenguaje de Definición de Datos**
 - Lenguaje que se usa para definir el esquema de la BD
- **LMD/DML – Lenguaje de Manipulación de Datos**
 - **Manipulación de Datos:**
 - Recuperación de información almacenada en la BD
 - Inserción de información nueva
 - Borrado de Datos
 - Modificación de la información
 - Permite acceder y manipular los datos organizados

Tipos de Usuarios

- Usuarios
- Administradores

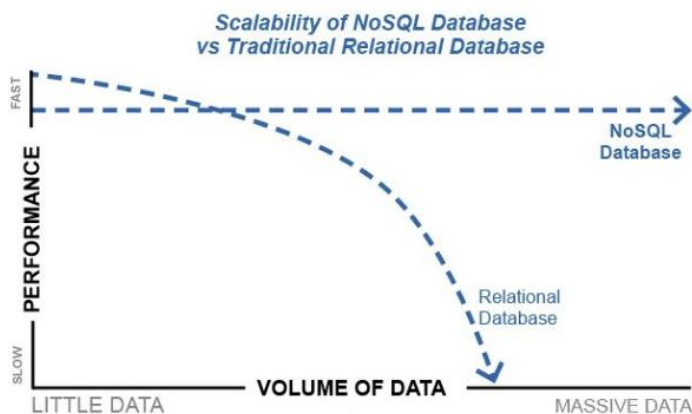
5 – BBDD no Relacionales

	Datos Estructurados	Datos No Estructurados
Display	Se puede mostrar en tablas	No se puede mostrar en tablas
Tipos de Datos	Numeros, fechas, cadenas	Imágenes, audios, videos, ficheros
Uso	20% de empresas	80% de empresas
Almacenamiento	Requiere menos almacenamiento	Requiere mas almacenamiento
Seguridad	Mas fácil de proteger	Mas complicado de proteger

BD Relacionales: La información se almacena en tablas

BD No Relacionales: La información se almacena en documentos

Rendimiento:



Tema 2 – Modelo Relacional y Algebra Relacional

1 – Reglas de Codd

Reglas de Codd: Reglas para determinar si una base de datos es relacional o no

1. **Regla de la información**
Toda la información se representa lógicamente como valores en tablas (relaciones).
2. **Acceso garantizado**
Todo dato debe ser accesible utilizando una combinación de nombre de tabla, columna y clave primaria.
3. **Tratamiento sistemático de valores nulos**
Los valores nulos deben tratarse de manera uniforme, sean datos faltantes, inaplicables o desconocidos.
4. **Catálogo activo basado en el modelo relacional**
La estructura de la base de datos (metadatos) también debe representarse en tablas accesibles mediante lenguaje relacional.
5. **Lenguaje completo de datos**
Debe haber un lenguaje que permita definir, consultar, modificar datos y controlar accesos (como SQL).
6. **Actualización de vistas**
Las vistas (consultas almacenadas) deben ser actualizables, siempre que tenga sentido hacerlo.
7. **Inserción, actualización y eliminación de datos**
Estas operaciones deben estar permitidas tanto en tablas como en vistas.
8. **Independencia física de datos**
Los cambios en el almacenamiento físico no deben afectar la forma en que se accede a los datos.
9. **Independencia lógica de datos**
Los cambios en la estructura lógica (como agregar una columna) no deben requerir cambios en las aplicaciones.
10. **Independencia de integridad**
Las reglas de integridad deben definirse en el modelo relacional, no en el código de la aplicación.
11. **Independencia de distribución**
El sistema debe funcionar correctamente independientemente de que los datos estén distribuidos en diferentes ubicaciones.
12. **Regla de la no subversión**
No debe haber forma de evitar las reglas del modelo relacional (como integridad o seguridad) usando lenguajes de bajo nivel.

2 – Estructura Modelo Relacional

Una base de datos relacional consiste en un conjunto de tablas con un nombre exclusivo

Cada fila de una tabla representa una relación entre un conjunto de valores

Atributos: Columnas de las tablas. Su nombre es la cabecera

Dominio: Conjunto de valores permitido para un atributo

Una tabla de n atributos debe ser subconjunto de $D_1 \times D_2 \times \dots \times D_n$

En vez de tabla y fila, se utilizan los términos relación y tupla.

El orden en el que aparecen las tuplas es irrelevante

Para todas las relaciones, el dominio de sus atributos ha de ser atómico

Dominio Atómico: Los elementos del dominio son indivisibles

El valor nulo es miembro de todos los dominios

El valor nulo provoca complicaciones en la definición de muchas operaciones

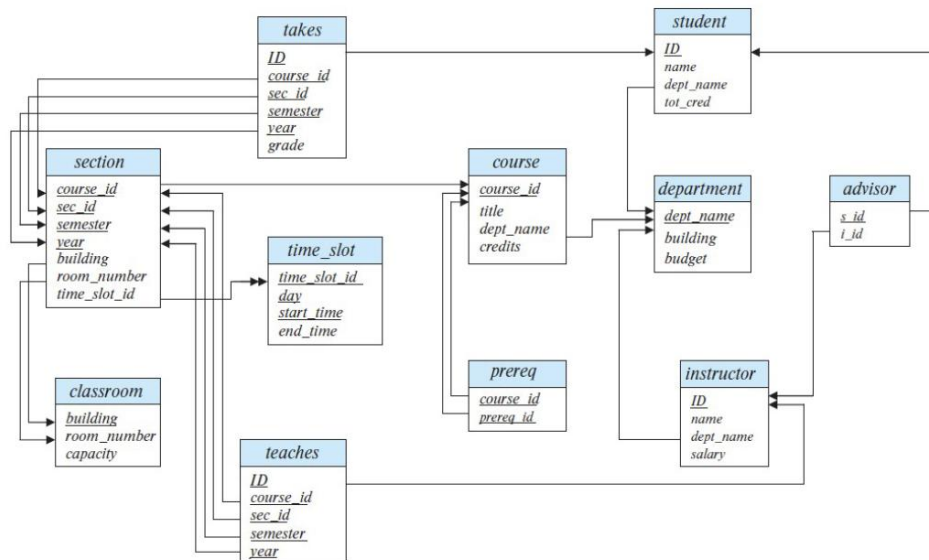
3 – Esquemas y Atributos del Modelo Relacional

Esquema de Una Relación:

- $A_1, A_2, \dots, A_n$ son los atributos
- $R = (A_1, A_2, \dots, A_n)$ es el esquema de la relación

Esquema de BD: Estructura lógica de la BD

Instancia de BD: Estado de los datos de la BD en un instante de tiempo dado



4 – Claves: Primarias y Foráneas

Debemos tener una manera de distinguir tuplas dentro de una relación dada

Clave Primaria:

- Verdadero identificador de cada registro
- Solo puede haber una clave primaria por tabla
- Los registros han de ser únicos
- Nos permite identificar de forma única cada tupla

Clave Foránea:

- Crea una relación entre tablas de forma que la columna se relaciona con la clave primaria de otra tabla
- Mantiene la integridad referencial

5 – Fases de Diseño BBDD

Diseño Conceptual:

- Esquema E/R
- Se describen la información de la base de datos y las relaciones entre los datos

Diseño Lógico:

- Parte del resultado del diseño conceptual
- Se monta el esquema de la BD
- Define las tablas, atributos, las relaciones, normalización

Diseño Físico:

- Implementación de la BD en memoria
- Se tiene cuenta el SGBD sobre el que se va a implementar

Cardinalidad:

- **1:1** – Cada elemento esta relacionado solo con 1 elemento de otra tabla (ej. Marido y Mujer). Cada tabla tiene la clave primaria de la otra como clave foránea
- **1: N/N:1** – Cada elemento está relacionado con 1 o varios elementos de otra tabla. Se guarda la clave primaria en la tabla N
- **N:N** – Varios elementos están relacionados con 1 o varios elementos de otra tabla. Se crea una nueva tabla que su clave primaria es la combinación de ambas claves foráneas

6 – Normalización

Normalización: Técnica de modelado para aplicar una serie de reglas a las relaciones obtenidas

Objetivos:

- Evitar Redundancia (Duplicación de datos)
- Simplificar los datos
- Garantizar la integridad referencial

Requerimientos:

- Cada tabla ha de tener un nombre único
- No puede haber 2 filas iguales
- Todos los datos de una columna han de ser iguales
- Se exige hasta la 3FN

Dependencia Funcional: Es una conexión entre uno o más atributos

Dependencia Funcional Reflexiva: Si Y esta incluido en X, entonces $X \rightarrow Y$

Dependencia Funcional Aumentativa: Si $X \rightarrow Y$, $XZ \rightarrow YZ$

Dependencia Funcional Transitiva: Si $X \rightarrow Y \rightarrow Z$, $X \rightarrow Z$

Primera Forma Normal: Sus atributos contienen valores atómicos y no hay registros repetidos

1. Duplicar registros con valores repetidos
 - a. Se soluciona eliminando el atributo que viola la condición
 - b. Se incluye un nuevo atributo que sea indivisible y se añade como clave primaria
2. Separar el atributo a otra tabla

Segunda Forma Normal: Esta en 1FN y todos los atributos tienen dependencia funcional completa respecto a todas las claves primarias. Aplican a tablas con claves primarias compuestas

1. Eliminar los atributos con dependencias incompletas
2. Crear tabla con los atributos de la que depende

Tercera Forma Normal: Esta en 2FN y todos los atributos que no son clave primaria no dependen transitivamente de esta

1. Separar en una tabla adicional los atributos que tiene dependencia transitiva y establecer como PK el campo que define la transitividad
2. Se añade este campo como FK

7 – Operaciones de Algebra Relacional

Algebra Relacional: Lenguaje procedural que consiste de un set de operaciones que cogen una o dos relaciones como entrada y producen una nueva relación como resultado

Select:

- Elige las tuplas que satisfacen el predicado
- $\sigma_p(r)$ donde p es el predicado y r es la relación
- Ej: $\sigma_{\text{dept_name} = \text{"Physics"}}(\text{instructor})$. Elige los instructores que trabajan en el departamento de Física.

And:

- Condición y
- $p \wedge q$ donde p y q son las condiciones
- Ej: $\sigma_{\text{dept_name} = \text{"Physics"} \wedge \text{salary} \leq 1000}(\text{instructor})$. Elige los instructores que trabajan en el departamento de Física y cobran menos de 1000.

Or:

- Condición o
- $p \vee q$ donde p y q son las condiciones
- Ej: $\sigma_{\text{dept_name} = \text{"Physics"} \vee \text{salary} \leq 1000}(\text{instructor})$. Elige los instructores que trabajan en el departamento de Física o cobran menos de 1000.

Not:

- Negación
- $\neg p$ donde p es la condición
- Ej: $\sigma_{\neg (\text{dept_name} = \text{"Physics"})}(\text{instructor})$. Elige los instructores no que trabajan en el departamento de Física.

Project:

- Devuelve solo los atributos indicados en p
- Π_p
- $\Pi_{\text{name}}(\sigma_{\text{dept_name} = \text{"Physics"}}(\text{instructor}))$. Saca el nombre de los instructores que trabajan en el departamento de Física.

Union

- Permite combinar 2 relaciones
- $r \cup s$ donde r y s son las relaciones a unir
- Para que sea valida:
 - r, s tiene que tener la misma aridad (n° atributos)
 - Los dominios han de ser compatibles
- Ej: $\Pi_{\text{course_id}}(\sigma_{\text{semester} = \text{"Fall"} \wedge \text{year} = 2017}(\text{section})) \cup \Pi_{\text{course_id}}(\sigma_{\text{semester} = \text{"Spring"} \wedge \text{year} = 2018}(\text{section}))$

Intersection

- Permite hacer la intersección de 2 relaciones
- $r \cap s$
- Mismas condiciones que la unión
- $\Pi_{\text{course_id}} (\sigma_{\text{semester} = \text{"Fall"} \wedge \text{year} = 2017} (\text{section})) \cap \Pi_{\text{course_id}} (\sigma_{\text{semester} = \text{"Spring"} \wedge \text{year} = 2018} (\text{section}))$

Set Difference

- Permite ver las tuplas que están en una relación y no en la otra
- $r - s$
- Mismas condiciones que la unión
- $\Pi_{\text{course_id}} (\sigma_{\text{semester} = \text{"Fall"} \wedge \text{year} = 2017} (\text{section})) - \Pi_{\text{course_id}} (\sigma_{\text{semester} = \text{"Spring"} \wedge \text{year} = 2018} (\text{section}))$

Join/Producto Cartesiano

- Permite combinar la información de 2 relaciones cualquiera
- $r \times s$
- $\sigma_{\text{instructor.id} = \text{teaches.id}} (\text{instructor} \times \text{teaches})$

Asignación

- Asigna nombres a operaciones
- Ej: $\text{Physics} \leftarrow \sigma_{\text{dept_name} = \text{'Physics'}} (\text{instructor})$

Tema 3 – Lenguaje SQL y SQL Avanzado

1 – Lenguaje SQL: DDL, DML, DCL, TCL

SQL: Lenguaje de programación diseñado para administrar y recuperar información de SGBD

Funciones de SQL:

- **DDL (Data Definition Language):**
Comandos para definir estructuras de datos: CREATE, DROP, ALTER
- **DML (Data Manipulation Language):**
Consultas y modificaciones de datos: SELECT, INSERT, DELETE, UPDATE.
- **DML (Data Manipulation Language):**
Consultas y modificaciones de datos: SELECT, INSERT, DELETE, UPDATE.
- **TCL (Transactional Control Language):**
Gestión de transacciones: COMMIT, ROLLBACK.

Tipos de Datos:

- **char(n):** Cadena de caracteres de longitud fija.
- **varchar(n):** Cadena de caracteres de longitud fija.
- **int, smallint:** Números enteros.
- **numeric(p,d):** Números de punto fijo con precisión.
- **real, double precision, float(n):** Números de punto flotante.

2 – Creación y Automatización de Tablas

Creación BBDD: CREATE DATABASE nombre_BD;

Creación Tabla:

```
CREATE TABLE nombre_tabla (  
    atributo1 tipo1,  
    atributo2 tipo2,  
    ...  
    PRIMARY KEY (atributo),  
    FOREIGN KEY (atributo) REFERENCES otra_tabla  
);
```

Insertión: INSERT INTO tabla VALUES (valores);

Eliminación: DELETE FROM tabla WHERE condición;

Modificación: UPDATE tabla SET atributo = valor WHERE condición;

Eliminar Tabla: DROP TABLE tabla;

Eliminar BD: DROP DATABASE nombre_BD;

Modificar Estructura: ALTER TABLE tabla ADD/DROP atributo tipo;

3 – Consultas

Estructura Básica:

SELECT atributos

FROM tablas

WHERE condición;

Cláusulas principales:

- **SELECT:** Proyecta columnas. Ejemplo: SELECT name FROM instructor;
- **WHERE:** Filtra filas. Ejemplo: SELECT name FROM instructor WHERE dept_name = 'Comp. Sci.';
- **FROM:** Especifica tablas. Ejemplo: SELECT * FROM instructor, teaches;
- **ORDER BY:** Ordena resultados. Ejemplo: SELECT name FROM instructor ORDER BY name DESC;
- **GROUP BY:** Agrupa resultados. Ejemplo: SELECT dept_name, AVG(salary) FROM instructor GROUP BY dept_name;
- **HAVING:** Filtra grupos. Ejemplo: SELECT dept_name, AVG(salary) FROM instructor GROUP BY dept_name HAVING AVG(salary) > 42000;
- **AS:** Renombrado
- **DISTINCT:** Elimina repetidos
- **LIKE:** Búsqueda en cadenas con formato específico
 - %: Reemplaza una subcadena
 - _: Reemplaza un subcarácter
 - Ej: “%dar”. Cadenas que tengan dar al final

Operaciones con conjuntos:

- **UNION:** Combina resultados eliminando duplicados.
- **INTERSECT:** Devuelve filas comunes.
- **EXCEPT:** Devuelve filas de la primera consulta que no están en la segunda.

Subconsultas:

SELECT name FROM instructor

WHERE salary > (SELECT AVG(salary) FROM instructor);

Nulos:

- **IS NULL:** Es nulo
- **IS NOT NULL:** No es nulo

Funciones de agregación:

- **AVG():** Media
- **MIN():** Mínimo
- **MAX():** Máximo
- **SUM():** Suma de valor
- **COUNT():** Numero de valores

Joins:

- **INNER JOIN:** Devuelve filas con coincidencias en ambas tablas.
- **LEFT JOIN:** Devuelve todas las filas de la tabla izquierda, aunque no tengan coincidencias.
- **RIGHT JOIN:** Devuelve todas las filas de la tabla derecha, aunque no tengan coincidencias.
- **FULL OUTER JOIN:** Devuelve todas las filas de ambas tablas.

4 – Comando Transaccionales y de Bloqueo

Transacciones:

- **START TRANSACTION:** Inicia una transacción.
- **COMMIT:** Confirma los cambios.
- **ROLLBACK:** Revierte los cambios.

Bloques de Tablas:

- Bloqueamos tablas para para el acceso concurrente a ellas
- **Bloqueo:** LOCK TABLES tabla WRITE;
- **Desbloqueo:** UNLOCK TABLES;

5 – Vistas. Tipos de Vistas

Vista: Tablas virtuales que muestran datos de tablas reales sin almacenarlos.

Ventajas:

- **Control De Accesos:** Se puede elegir que información compartir con los usuarios
- **Mejoras de Rendimiento:** Se pueden crear queries a partir de vistas
- **Pruebas Seguras:** Ofrecen un entorno seguro para que los desarrolladores no afecten a la información real
- **Reusabilidad de Consultas:** Simplifica queries
- **Mantenimiento de la Integridad:** Garantiza que no se rompan las tablas.

Tema 4 – Acceso a MySQL, Almacenamiento e Indexado

1 – Formas de Acceso a BBDD

Métodos de interacción con bases de datos

1. **Consola del DBMS:** Ejecución directa de comandos SQL en la interfaz de línea de comandos.
2. **Scripts:** Archivos con comandos SQL que se ejecutan automáticamente.
3. **Herramientas gráficas:** Interfaces como MySQL Workbench o phpMyAdmin.
4. **Programas externos:** Conexión mediante lenguajes de programación como Python, PHP, Java, etc.

2 – Acceso por Consola

Comandos básicos en MySQL:

- **Conexión:** `mysql -u usuario -p`
- **Mostrar Bases De Datos:** `SHOW DATABASES;`
- **Seleccionar Base de Datos:** `USE nombre_bd;`
- **Mostrar Tablas:** `SHOW TABLES;`
- **Consultar datos:** `SELECT * FROM tabla;`

3 – Interacción Mediante Scripts

Se escribe todos los comandos en un script

Ejecución Script: `mysql -u usuario -p nombre_bd < script.sql`

4 – Interacción a Traves de Lenguajes de Programación

Python:

- **Librería:** MySQLdb
- **Conexión:** `conn = MySQLdb.connect(host='localhost', user='root', passwd='contraseña', db='nombre_bd')`
- **Ejecución Query:** `cursor.execute(query)`
- **Coger Datos:** `cursor.fetchall();`

PHP:

- **Conexión:** `$conn = mysqli_connect($servername, $username, $password, $dbname);`
- **Ejecución Query:** `mysqli_query($conn, $sql)`
- **Coger Datos:** `mysqli_fetch_assoc($result)`

5 – Tipos de Almacenamiento

Clasificación por persistencia:

- **Volátil:** Memoria cache y memoria principal (se pierden datos sin energía).
- **No volátil:** Discos duros, memorias flash, etc. (datos persistentes).

Parámetros Relevantes:

- **Velocidad de Acceso:** Cuanto tiempo tarda el sistema en leer o escribir información (Mb/s)
- **Coste de Almacenamiento:** Coste de Almacenar la información (€/GB)
- **Fiabilidad:** Se mide mediante el tiempo medio entre fallos (MTBF)

Memorias:

- **Cache:** Memoria muy rápida y cara. Poco espacio y volátil
- **Memoria Principal:** Memoria que usa el sistema para todas sus operaciones. Mayor que la cache pero no lo suficiente para tener un BD. Cara y Volátil
- **Memoria Flash:** Memoria no volátil con tiempos de lectura similares a la memoria principal. Tiempos de escritura mucho más largos

Discos:

- **Discos Magnéticos:** Memoria no volátil y relativamente barata, pero con velocidad de acceso inferior. Para actuar sobre los datos hay que hacerlo sobre memoria. Mucha latencia
- **Discos Ópticos:** Mucha mas capacidad que los discos magnéticos pero peores prestaciones de lectura/escritura. Son los más económicos
- **Cintas Magnéticas:** Usadas para hacer copias de seguridad. Mucho mas barata pero mucho mas lenta ya que el acceso es secuencial.

Sistemas RAID:

- Las técnicas de organización de discos denominadas matrices redundantes de discos independientes se usan para lograr un mejor rendimiento y fiabilidad de los sistemas de almacenamiento.
- **Fiabilidad:** Se incrementa aplicando redundancia
- **Prestaciones:** Se distribuyen los datos de manera equitativa entre las unidades que lo forman. Se duplica la velocidad de datos
- RAID 0: Striping (mejora rendimiento, sin redundancia).
- RAID 1: Mirroring (redundancia completa).
- RAID 5: Striping con paridad (balance entre rendimiento y redundancia).

Sistemas de almacenamiento en red:

- **DAS (Direct Attached Storage):** Almacenamiento conectado directamente a un servidor.
- **NAS (Network Attached Storage):** Almacenamiento accesible a través de una red.
- **SAN (Storage Area Network):** Red dedicada de alta velocidad para almacenamiento.

6 – Jerarquía

Niveles

1. **Primario:** Memoria cache y RAM (rápido, volátil).
2. **Secundario:** Discos duros y memorias flash (no volátil, acceso moderado).
3. **Terciario:** Cintas magnéticas (lento, para copias de seguridad).

7 – Indexado

Índice: Estructuras que mejoran la velocidad de las consultas al evitar escaneos completos de tablas.

Operaciones:

- **Crear:** CREATE INDEX nombre_indice ON tabla(columna);
- **Eliminar:** DROP INDEX nombre_indice ON tabla;
- **Mostrar:** SHOW INDEX FROM tabla;

Tipos de índices

- **Índice PRIMARY:** Clave primaria (almacenado con los datos).
- **Índices secundarios:** Mejoran el rendimiento de consultas específicas.

Tema 5 – Administración de Bases de Datos

MySQL

1 – Lenguaje SQL: DCL y DDL

DCL:

- **GRANT:** Otorga permisos. GRANT SELECT ON bd.tabla TO usuario@host;
- **REVOKE:** Elimina permisos. REVOKE INSERT ON bd.tabla FROM usuario@host;

DDL:

- **CREATE:** Crea objetos (BBDD, tablas, vistas). CREATE TABLE empleados (id INT PRIMARY KEY, nombre VARCHAR(50));
- **ALTER:** Modifica estructuras existentes. ALTER TABLE empleados ADD COLUMN email VARCHAR(100);
- **DROP:** Elimina objetos. DROP DATABASE prueba;

2 – Usuarios y Privilegios

Operaciones Basicas:

- **Crear Usuario:** CREATE USER 'nuevo_usuario'@'localhost' IDENTIFIED BY 'claveSegura123';
- **Mostrar usuarios:** SELECT user, host FROM mysql.user;
- **Cambiar contraseña:** ALTER USER 'usuario'@'localhost' IDENTIFIED BY 'nuevaClave456';
- **Renombrado:** RENAME USER 'nuevo_usuario'@'localhost' TO 'nuevo_usuario2'@'localhost';

Asignación de Privilegios:

- **GRANT SELECT, INSERT ON bd.* TO 'usuario'@'localhost';**
- **SELECT:** Permite lectura de datos
- **INSERT:** Permite insertar datos
- **UPDATE:** Permite modificar datos

Gestión de Roles:

- **Crear Rol:** CREATE ROLE rol_lectura;
- **Asignar Privilegios:** GRANT SELECT ON bd.* TO rol_lectura;
- **Asignar rol a usuario:** GRANT rol_lectura TO 'usuario'@'localhost';

3 – Operativa Básica de MySQL

Gestión del servicio:

- **Iniciar servicio:** service mysql start
- **Detener servicio:** service mysql stop
- **Reiniciar servicio:** service mysql restart
- **Ver estado:** service mysql status

4 – Copias de Seguridad

Tipos de backup

- **Completo:** mysqldump -u root -p --all-databases > backup_completo.sql
- **Base de datos específica:** mysqldump -u root -p nombre_bd > backup_bd.sql
- **Solo estructura:** mysqldump -u root -p --no-data nombre_bd > estructura.sql
- **Solo datos:** mysqldump -u root -p --no-create-info nombre_bd > estructura.sql

5 – Restauración de Copias

Métodos de restauración:

- **Desde línea de comandos:** mysql -u root -p nombre_bd < backup_bd.sql
- **Desde consola MySQL:** SOURCE /ruta/backup_bd.sql;

6 – Optimización/Tuning

Ciclo de optimización (TLC):

- **Medir:** Identificar cuellos de botella (CPU, memoria, I/O).
- **Analizar:** Determinar causas de ineficiencia.
- **Cambiar:** Ajustar configuración o esquema.
- **Probar:** Verificar impacto de cambios.
- **Implementar:** Aplicar modificaciones definitivas.

Técnicas clave:

- **Índices:** Mejoran velocidad de consultas.
CREATE INDEX idx_nombre ON empleados(nombre);
- **Optimización de queries:**
 - Usar EXPLAIN para analizar consultas.
 - Evitar SELECT *.
 - Preferir JOIN sobre subconsultas.
- **Ajuste de configuración:**
 - query_cache_size: Almacena resultados de queries frecuentes.
 - innodb_buffer_pool_size: Memoria para caché de InnoDB.

Tema 6 – Aspectos Avanzados de Bases de Datos

1 – Lenguajes Procedurales de la Base de Datos

Lenguaje Procedural	Lenguaje No Procedural
Especifica "qué hacer" y "cómo hacerlo" (ej: C, Java)	Solo especifica "qué hacer" (ej: SQL, LISP)
Ejecución secuencial	Basado en funciones
Alta eficiencia	Menor eficiencia
Ideal para procesos complejos	Ideal cuando el tiempo es crítico

2 – Procedimientos Almacenados

Procedimientos:

DELIMITER \$\$

```
CREATE PROCEDURE listar_productos(IN gama VARCHAR(50))
```

```
BEGIN
```

```
    SELECT * FROM producto WHERE producto.gama = gama;
```

```
END $$
```

- **Invocación:** CALL procedimiento
- **Mostrar Procedimientos:** SHOW PROCEDURE

Funciones:

```
CREATE FUNCTION contar_productos(gama VARCHAR(50))
```

```
RETURNS INT
```

```
BEGIN
```

```
    RETURN (SELECT COUNT(*) FROM producto WHERE producto.gama = gama);
```

```
END $$
```

- Siempre devuelve un valor
- **Mostrar Funciones:** SHOW FUNCTION

Triggers:

```
CREATE TRIGGER trigger_check_nota  
BEFORE INSERT ON alumnos  
FOR EACH ROW  
BEGIN  
    IF NEW.nota < 0 THEN SET NEW.nota = 0;  
    ELSEIF NEW.nota > 10 THEN SET NEW.nota = 10;  
    END IF;  
END $$
```

- Se ejecutan ante eventos
- **Mostrar Triggers:** SHOW TRIGGERS;

3 – SQL Inyection

SQL Inyection: Ataque que explota vulnerabilidades en consultas SQL mal sanitizadas, permitiendo ejecutar código no autorizado.

Ejemplo: SELECT * FROM usuarios WHERE usuario = ' + inputUsuario + ' AND clave = ' + inputClave + '";

Solución: Usar consultas preparadas (PreparedStatement en Java) o ORMs

4 – Bases de Datos NoSQL

Relacionales	NoSQL
Estructura rígida (tablas)	Esquema flexible (documentos, grafos)
ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad)	BASE (Basic Availability, Soft state, Eventual consistency)
Escalabilidad vertical	Escalabilidad horizontal
Ideal para datos estructurados	Ideal para Big Data y datos semiestructurados

Ventajas	Desventajas
Masivamente escalables	Capacidad de Interrogación Limitada
Alta Disponibilidad	Consistencia eventual
Bajo Coste	No estandarizado, no portable
Elasticidad Predecible	Desarrollo mas complejo
Esquema Flexible	Carecen de herramientas de control de acceso

5 – Clasificación de Bases de Datos NoSQL

Documentales (MongoDB, CouchDB)

Clave-Valor (Redis, DynamoDB)

Columnas (Cassandra)

Grafos (Neo4j)

6 – MongoDB

Características:

- **Documentos JSON/BSON:** Estructura flexible.
- **Alta disponibilidad:** Réplicas automáticas.
- **Escalabilidad horizontal:** Sharding distribuido.
- **Consultas ad-hoc:** Indexación avanzada

Operaciones Básicas (CRUD):

- **Crear:** `db.posts.insertOne({
 title: "Post 1",
 category: "News",
 likes: 10
});`
- **Consultar:** `db.posts.find({ category: "News" });`
- **Actualizar:** `db.posts.updateOne(
 { title: "Post 1" },
 { $set: { likes: 15 } }
);`
- **Eliminar:** `db.posts.deleteOne({ title: "Post 1" });`